

Characterizing Single-GPU Decoder-Only Transformer Inference: Precision, Compilation, Batching, and Model Size on an RTX 5070

Ben Karlsberg

Abstract

We present a controlled single-GPU study of decoder-only transformer inference with three open SmolLM models (135M, 360M, and 1.7B parameters) on an NVIDIA GeForce RTX 5070. For the 135M model, we measure synchronized end-to-end `generate()` latency and tokens-per-second across a 32-cell matrix spanning data type (FP16, BF16), batch size (1, 4), prompt length (64, 256), generation length (32, 128), and execution mode (eager vs. `torch.compile`). BF16 reduced latency relative to FP16 in every paired comparison (median latency ratio 0.90). A batch-size sweep (1 to 256) locates the latency-throughput knee: latency stays near 2.3 s up to batch 32 while throughput rises about $31\times$, after which latency grows roughly linearly with batch size and throughput rises only from 2,222 tok/s at batch 64 to 2,467 tok/s at batch 256. Repeating the sweep on the 360M and 1.7B models shows the knee moving to smaller batches as the model grows (batch 16 for 360M; no flat region for 1.7B), and that per-token latency at batch 1 does not follow parameter count: the 1.7B model takes about 40% less time per decode step than the 135M model. Decode-step profiling explains this: at batch 1 the GPU is idle 39–87% of each step while the host issues 990–1,790 kernels, and on this Windows PyTorch build, which lacks FlashAttention, grouped-query attention in the smaller models falls back to a math path costing about a fifth of their step time. For the 1.7B model, the default growing KV cache fragments PyTorch’s allocator enough that batch 64 reserves 24 GB for 8 GB of live data, and the Windows driver’s default system-memory fallback moves the excess into host memory; a preallocated static cache avoids this and reaches 2,044 tok/s. Generation length dominated end-to-end time. `torch.compile` gave little systematic gain (median eager/compile latency ratio 1.02, range 0.92–1.22).

1 Introduction

Autoregressive transformer inference [1] is commonly deployed with mixed precision, compiler stacks, and request batching. Practitioners must choose among these options without assuming that every optimization helps every workload. This paper reports a small, reproducible characterization on one consumer GPU using the open-source `goti` harness accompanying this work. Our goal is empirical clarity at a fixed hardware point—a 135M-parameter model for the factorial matrix and the first batch sweep, and two larger models from the same family for a model-size sweep—not a claim of state-of-the-art serving performance.

Serving systems batch requests on the premise that small-batch decoding leaves the GPU underused [29, 14, 19]. How far batching stays free before latency rises depends on the model and the device, and is rarely reported for small models on consumer GPUs through the standard Hugging Face `generate()` path. Our contributions are (i) a 32-cell factorial measurement of precision, batch size, prompt length, generation length, and `torch.compile` for a 135M-parameter model on an RTX 5070; (ii) a batch-size sweep from 1 to 256 that locates the latency-throughput knee, with an explicit account of an excluded noisy pass; (iii) a repetition of the sweep across three model sizes and two KV-cache implementations, including a memory-fragmentation failure mode that is invisible to allocated-memory metrics; (iv) a per-step profile of eager decoding that attributes small-batch step time to host-side kernel issue and identifies a math-backend attention fallback for grouped-query attention on this Windows PyTorch build; and (v) an open harness that

records the configuration, environment, and peak allocated memory of every run, and additionally reserved and driver-reported memory for the model-size runs. The batch-size throughput plateau and latency knee themselves have been reported before for small models on datacenter GPUs [21, 22]; what is new here is a measurement on a consumer Blackwell GPU (GeForce RTX 5070, 12 GB), the observation that the knee moves to smaller batches as model size grows, the finding that per-step latency at batch 1 does not track parameter count, a decode-step profile that explains it, and a failure mode in which the growing KV cache (DynamicCache) fragments PyTorch’s allocator until the Windows driver falls back to system memory, together with a static-cache mitigation.

2 Related work

Batching characterizations of small models. The closest prior work is by Recasens et al. [21], who served OPT models from 125M to 13B parameters with vLLM on A100 GPUs and found that small models reach a throughput plateau within a single accelerator: for OPT-125M and OPT-1.3B, batches beyond 128 bought marginal throughput at a considerable latency cost. A follow-up study [22] profiled large-batch decoding on an H100 and reported that it remains memory-bound, with DRAM bandwidth saturated in the attention kernels rather than compute, and that CPU-side time between decode steps, during which no GPU kernel runs, reaches up to about 30% of decode time for OPT-1.3B. Broader characterizations benchmark 7B–70B models across accelerators and serving frameworks (LLM-Inference-Bench [8]), analyze prefill/decode heterogeneity down to warp stalls on A100 GPUs [26], and relate decode-phase latency and energy to GPU frequency scaling on a workstation Blackwell GPU [15]. These studies use serving engines on datacenter or workstation GPUs; we measure the knee on a 12 GB consumer GPU through the Hugging Face `generate()` path, where it interacts with model shape and with KV-cache allocation.

Serving systems and batching. Orca [29] introduced iteration-level scheduling so that new requests can join a running batch, and vLLM [14] stores the KV cache in fixed-size pages so that more sequences fit in memory at once. Sarathi-Serve [3] studies the throughput–latency tradeoff created when prefill and decode share a batch. Pope et al. [19] model inference cost as a function of batch size, memory bandwidth, and partitioning on TPUs, and DeepSpeed-Inference [6] and FlexGen [25] target models too large for a single device or its memory. These systems motivate batching; our sweep measures where its benefit ends for a small model, using static batches rather than a serving engine.

Kernels and attention. FlashAttention [10, 9] reduces attention memory traffic by tiling the computation, and PyTorch’s scaled-dot-product attention dispatches to fused kernels of this kind. Multi-query attention [24] shrinks the KV cache read at every decode step. Our harness uses the stock attention path and does not modify kernels; on our platform that path has no FlashAttention backend, which matters for grouped-query attention (Section 4.5).

Precision, quantization, and compilation. FP16 mixed precision [16] and BF16 [13] were introduced mainly for training speed and numerical range; their relative latency at inference on a given GPU is an empirical question that we measure directly. Weight quantization such as LLM.int8() [11] and GPTQ [12] further reduces memory traffic; we leave it to future work. `torch.compile` [7] captures Python programs as graphs (TorchDynamo) and generates fused kernels (TorchInductor); we evaluate its default mode.

Benchmarking methodology. MLPerf Inference [23] standardizes inference measurement across systems, and the roofline model [27] relates achieved performance to compute and bandwidth limits. Our study is

narrower: a controlled factor sweep on one consumer GPU, intended as a reproducible reference point rather than a benchmark suite.

3 Method

3.1 Software stack

Experiments use PyTorch [2] with CUDA 12.8 bindings (`torch 2.11.0+cu128`), Hugging Face Transformers [28] for model loading, and scaled-dot-product attention via PyTorch SDPA backends. The workload is `greedy model.generate()` with KV caching enabled. Synthetic token ids form prompts so logging of decoded text cannot contaminate timed regions.

3.2 Metrics and timing protocol

For each configuration we run a warmup phase followed by measured iterations. Each iteration records GPU-synchronized end-to-end latency of `generate()`, peak CUDA memory allocated, and tokens-per-second derived from generated token count and wall time. We report distributional summaries (mean, selected percentiles) rather than a single timing. Cold-start model load is outside the measured loop; `torch.compile` configurations still include steady measured iterations after warmup, but compilation cost can affect short windows and is treated as part of the system under test.

3.3 Experimental matrix

Model: HuggingFaceTB/SmolLM-135M [5], a 135M-parameter decoder-only language model. Factors: `dtype` $\in \{\text{fp16}, \text{bf16}\}$, batch size $\in \{1, 4\}$, prompt length $\in \{64, 256\}$, max new tokens $\in \{32, 128\}$, compile $\in \{\text{false}, \text{true}\}$. This yields 32 cells. Hardware: a single desktop workstation with an NVIDIA GeForce RTX 5070 (12 GB GDDR7, 672 GB/s rated memory bandwidth [17]; NVIDIA 610-series driver with its default settings) running Windows 11.

4 Results

All 32 cells completed with no OOM events. The lowest mean end-to-end latency was 586 ms (BF16, batch 4, prompt 256, gen 32, compile enabled; ≈ 218 tok/s). The highest was 2855 ms (FP16, batch 4, prompt 64, gen 128, eager; ≈ 180 tok/s).

4.1 Precision

Figure 1 compares FP16 and BF16 latency for every matched non-dtype factor. BF16 was faster in all 16 pairs. The median BF16/FP16 latency ratio was 0.895 (mean 0.890).

4.2 Batching and throughput

Figure 2 shows tokens-per-second versus batch size for FP16 eager runs. Moving from batch 1 to batch 4 increased throughput from 41–49 tok/s to 180–189 tok/s. End-to-end latency did not rise with batch size in this range: going from batch 1 to batch 4 raised throughput without a latency penalty. With only two batch sizes, however, the matrix cannot show where that stops, so we ran a dedicated sweep (Section 4.3).

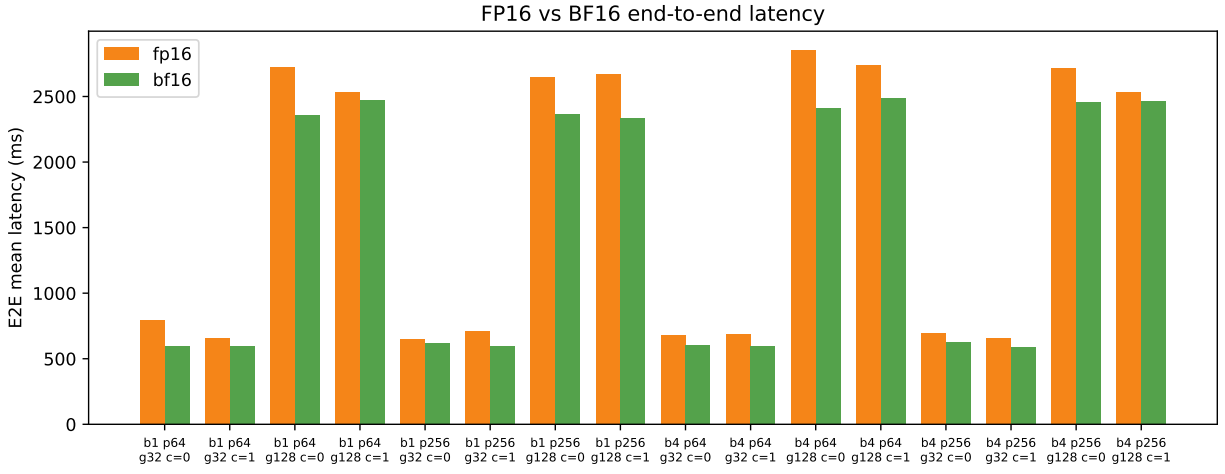


Figure 1: End-to-end mean latency for FP16 vs. BF16 across matched matrix cells.

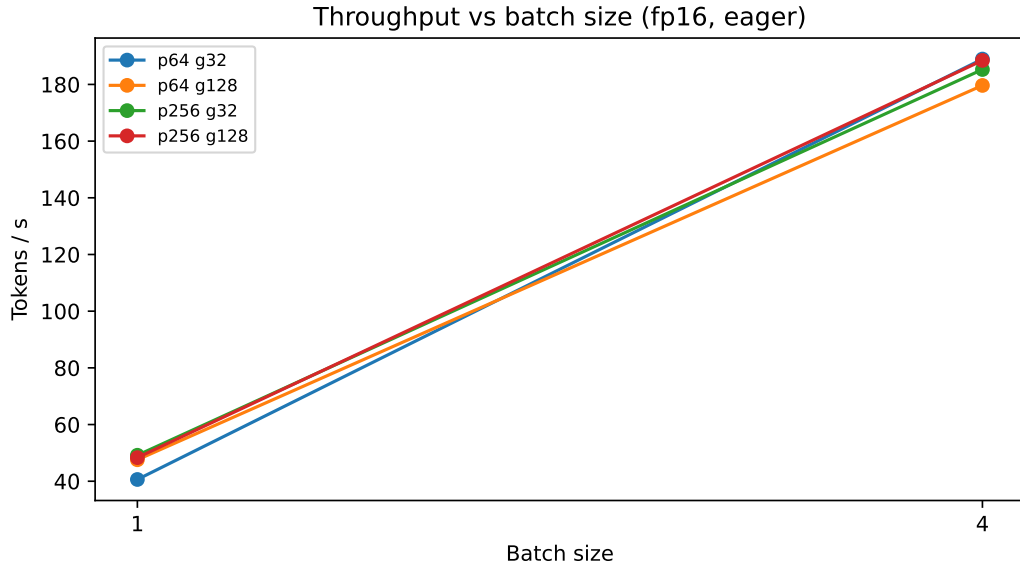


Figure 2: Throughput versus batch size (FP16, eager).

4.3 Batch-size sweep: the latency–throughput tradeoff

To locate where batching stops being free, we swept batch size over $\{1, 2, 4, \dots, 256\}$ with all other factors fixed (BF16, eager, prompt 256, 128 new tokens; 2 warmup and 5 measured iterations per cell). We ran the sweep three times. The first pass was inconsistent with the matrix (batch 1 took 3.4 s versus 2.37 s in the matrix, and batch 256 was faster than batch 128), which we attribute to host-side variability on the workstation during that pass; it is excluded from the aggregate but kept in the released analysis file. The second pass ran batch sizes in descending order and the third in ascending order; they agreed within 2% at every batch size, and Table 1 and Figure 3 report their mean. No cell ran out of memory; peak allocated memory was 4.0 GB at batch 256.

Batch	Latency (ms)	Throughput (tok/s)	Peak memory (MB)
1	2303	56	283
2	2314	111	299
4	2354	218	328
8	2335	438	386
16	2317	884	502
32	2336	1754	733
64	3687	2222	1197
128	6827	2401	2126
256	13286	2467	3980

Table 1: Batch-size sweep (BF16, eager, prompt 256, 128 new tokens), mean of two passes.

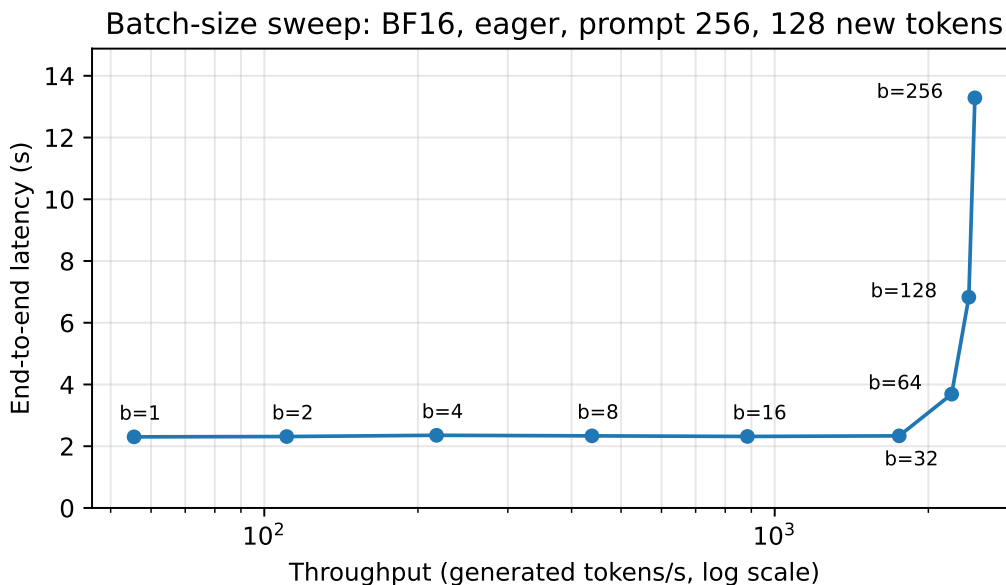


Figure 3: Latency versus throughput across the batch-size sweep. Each point is one batch size; the curve is flat up to batch 32 and then turns sharply upward.

Two regimes are visible. From batch 1 to batch 32, end-to-end latency stays between 2.30 and 2.35 s (about 18 ms per decode step) while throughput rises from 56 to 1,754 tok/s. In this regime a decode step

for this 135M-parameter model does not occupy the GPU, so per-step time is set by host-side costs that do not grow with batch size, and additional sequences are nearly free. Beyond batch 32 the GPU becomes the bottleneck: latency grows roughly in proportion to batch size (3.7 s at 64, 6.8 s at 128, 13.3 s at 256) while throughput rises only from 2,222 tok/s at batch 64 to 2,467 tok/s at batch 256. For a latency target, the useful operating point on this hardware is therefore around batch 32; batch 64 buys about 27% more throughput at a 58% latency cost, and larger batches add little throughput. Profiling (Section 4.5) supports this reading; beyond the knee the GPU is busy more than 80% of the step, but its time goes mostly to a math-backend attention path rather than to streaming weights or the KV cache, unlike the DRAM-bandwidth-bound fused attention kernels profiled on an H100 with vLLM [22].

4.4 Model size and KV-cache allocation

To test whether the knee is a property of the GPU or of the model, we repeated the sweep with two larger members of the same family, SmolLM-360M and SmolLM-1.7B [5], under the same settings. Table 2 lists their shapes; the two smaller models are deep and narrow and use grouped-query attention [4], while the 1.7B model has fewer, wider layers and full multi-head attention. Each configuration again ran twice, once in descending and once in ascending batch order; the two passes agreed to within about 1.5% at every valid cell. These runs came from a later session, after a change to the workstation’s power plan, so we also re-ran the 135M sweep in it. It reproduced the earlier sweep within 1.1% at every batch size up to 128 and took 4.5% less time at batch 256; Table 3 uses the re-run so that all three models share one session. For these runs the harness additionally recorded PyTorch’s reserved memory, the driver-reported memory in use, and the Windows shared-GPU-memory counter, which reveals allocations that the driver has placed in system RAM instead of failing; the earlier matrix and batch-sweep runs recorded only peak allocated memory.

Model	Layers	Hidden	Query heads	KV heads	BF16 weights
SmolLM-135M	30	576	9	3	0.27 GB
SmolLM-360M	32	960	15	5	0.72 GB
SmolLM-1.7B	24	2048	32	32	3.4 GB

Table 2: Architectures of the three models (from their published configurations).

Batch	135M		360M		1.7B dyn.		1.7B static	
	ms	tok/s	ms	tok/s	ms	tok/s	ms	tok/s
1	2308	55	2457	52	1379	93	1706	75
2	2332	110	2437	105	1514	169	1732	148
4	2365	216	2430	211	1614	317	1748	293
8	2330	440	2451	418	1801	569	1848	554
16	2341	875	2481	826	2233	917	2154	951
32	2333	1756	3504	1169	3232	1268	2751	1489
64	3680	2226	6301	1300	spilled [†]		4008	2044
128	6754	2426	11544	1419	–	–	–	–
256	12684	2583	22282	1471	–	–	–	–

Table 3: End-to-end latency and throughput across model sizes; “dyn.” is the default growing KV cache (BF16, eager, prompt 256, 128 new tokens; mean of two passes). [†]About 13 GB was placed in system memory and latency rose to 35 s; the cell is excluded. Dashes mark batch sizes that were not run because they cannot fit in 12 GB.

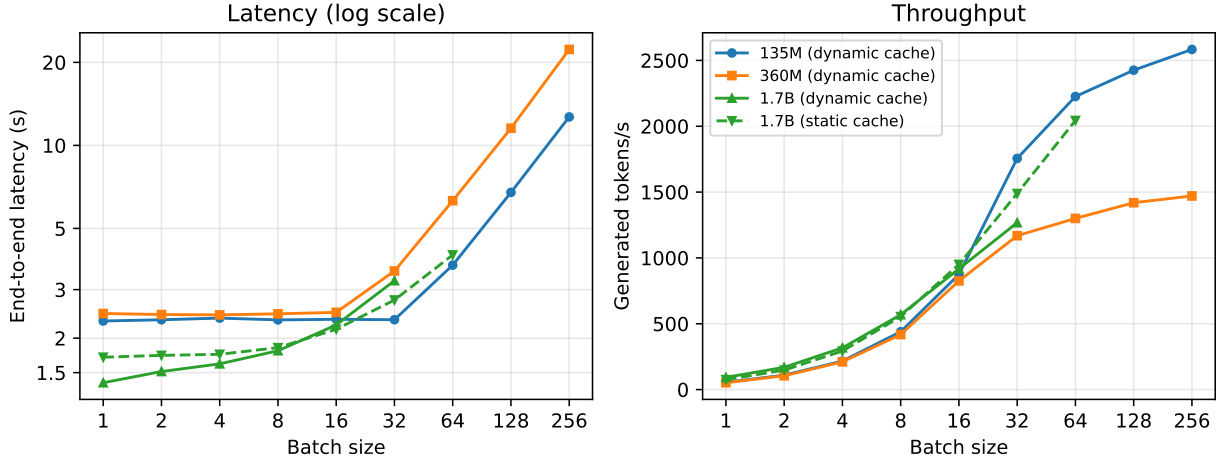


Figure 4: Latency (left, log scale) and throughput (right) versus batch size for three model sizes. The dashed line is the 1.7B model with a static KV cache; the spilled 1.7B dynamic-cache cell at batch 64 is omitted.

The knee moves left as the model grows. Latency for the 360M model stays between 2.43 and 2.48 s through batch 16 and then rises to 3.50 s at batch 32, one doubling earlier than the 135M model. The 1.7B model has no flat region: relative to batch 1, latency is 17% higher at batch 4, 31% higher at batch 8, and 62% higher at batch 16. The regime in which extra sequences are nearly free therefore shrinks with model size and has disappeared by 1.7B parameters on this GPU.

Per-step latency does not follow parameter count. At batch 1 the three models take 18.0, 19.2, and 10.8 ms per decode step respectively. The 360M model has $2.7\times$ the parameters of the 135M model but takes only 6% longer per step, and the 1.7B model, with $13\times$ the parameters, takes 40% less time per step (about $1.67\times$ the speed). Per layer, the two smaller models take about 0.60 ms and the 1.7B model about 0.45 ms. Reading the 1.7B model’s 3.4 GB of weights at the card’s rated 672 GB/s memory bandwidth [17] would take about 5 ms, roughly half of the measured step. Section 4.5 shows that at batch 1 all three models are host-bound, that the two grouped-query models [4] also pay for a math-backend attention fallback, and that weight streaming accounts for most of the 1.7B model’s GPU time without setting its step time. The same pattern holds for throughput: the 360M model saturates at 1,471 tok/s, below both the 135M model (2,583) and the 1.7B model with a static cache (2,044 at batch 64).

KV-cache allocation and system-memory fallback. The default KV cache in Transformers grows by one position per decode step (the `DynamicCache` class). For the two smaller models, whose grouped-query caches are small, PyTorch’s reserved memory never exceeds allocated memory by more than 18% (at most about 0.25 GB). For the 1.7B model, whose full multi-head cache is about 75 MB per sequence at these lengths, the ratio of reserved to allocated memory grows from 1.06 at batch 1 to 1.29 at batch 16, 1.72 at batch 32 (9.7 versus 5.6 GB), and 3.06 at batch 64 (24.4 versus 8.0 GB). At batch 64 the reserved memory exceeds the card’s 12 GB. NVIDIA’s Windows drivers include a documented CUDA system-memory fallback, introduced in driver 536.40, that places allocations in shared system memory when GPU memory is nearly exhausted instead of failing; since driver 546.01 it can be disabled per application through the `CUDA – Sysmem Fallback Policy` setting (*Prefer No Sysmem Fallback*) [18]. Our runs used the driver’s default policy, so instead of an out-of-memory error, about 13 GB was placed in shared memory and the batch-64 cell took 35.1 s. The fallback itself is documented driver behavior; what allocated-memory metrics hide is the

cause, namely the growth of reserved memory under the dynamic cache, which pushes the process past the device limit while allocated memory would have reported this cell as a comfortable 8 GB run. PyTorch’s expandable-segments allocator option, the usual remedy for fragmentation, is not supported on this platform, and garbage-collection thresholds did not prevent the spill in a preliminary test. Preallocating the cache for the full sequence (`cache_implementation="static"`, run eagerly) removes the growth: batch 64 reserves 10.7 GB, runs in 4.0 s, and delivers 2,044 tok/s, versus 1,268 tok/s at batch 32, the largest batch the growing cache could run. The static cache takes 24% longer at batch 1 but 4% less time at batch 16 and 15% less at batch 32 (about $1.17\times$ the throughput). Profiling (Section 4.5) attributes the batch-1 penalty mainly to extra kernels per step and the batch-32 gain to avoiding the growing cache’s concatenation copies.

4.5 Where decode-step time goes

To measure what the sections above could only infer, we profiled single decode steps below and above each model’s knee (Table 4). A script (`scripts/profile_decode.py`) loads each model in BF16 (eager, SDPA), prefills the same 256-token synthetic prompt, and runs a manual greedy loop over a `DynamicCache` (or a static cache) that mirrors `generate()`. After 8 warmup steps it times 32 steps synchronized after each (wall time) and 32 unsynchronized (host dispatch time), then records 10 steps with `torch.profiler` (CUPTI) to count kernels and compute GPU busy time as the union of kernel and memory-operation intervals; it also times `generate()` for the same prompt. Idle fractions use the unprofiled wall time, since the profiler lengthens steps by $1.34\text{--}1.40\times$ below the knee, excluding the first passes of the two starred cells in Table 4 ($2.05\times$ and $1.06\times$), and by at most $1.15\times$ above it. Each cell ran twice, the second pass in reverse order.

Model	Cache	Batch	Wall (ms)	GPU busy (ms)	GPU idle	Kernels/step	Weight BW (GB/s)
135M	dynamic	1	21.01 $\pm$ 1.10*	2.73 $\pm$ 0.01	87%	1680	98 (15%)
		32	19.88 $\pm$ 0.26	8.50 $\pm$ 0.01	57%	1680	
		128	44.14 $\pm$ 0.03	36.56 $\pm$ 0.00	17%	1680	
360M	dynamic	1	24.53 $\pm$ 3.26*	4.17 $\pm$ 0.00	83%	1790	173 (26%)
		16	21.03 $\pm$ 0.00	7.92 $\pm$ 0.03	62%	1790	
		64	41.09 $\pm$ 0.01	33.19 $\pm$ 0.04	19%	1822	
1.7B	dynamic	1	11.89 $\pm$ 0.02	7.24 $\pm$ 0.00	39%	990	473 (70%)
		16	14.87 $\pm$ 0.01	11.85 $\pm$ 0.01	20%	990	
		32	19.87 $\pm$ 0.03	16.55 $\pm$ 0.01	17%	1014	
1.7B	static	1	13.76 $\pm$ 0.09	7.52 $\pm$ 0.00	45%	1144	455 (68%)
		32	17.20 $\pm$ 0.01	13.21 $\pm$ 0.01	23%	1170	

Table 4: Decode-step profile (BF16, eager, SDPA, prompt 256; manual greedy loop). All values are means of two passes; for wall and GPU busy, $\pm$ gives half the range between the passes. Wall: median synchronized step time. GPU busy: union of kernel and memory-operation time per step. Idle: $1 - \text{busy}/\text{wall}$. Weight BW: BF16 weight bytes divided by GPU busy time, batch 1 only (percentage of the rated 672 GB/s). *Anomalous first pass (22.1 and 27.8 ms versus 19.9 and 21.3 ms); the second matches the separate GQA runs below (19.9–20.0 and 21.4–21.6 ms). Kernel counts are identical across passes.

Below the knee the GPU mostly waits for the host. Each step issues exactly 55 kernels per decoder layer plus 30 outside the layers for the two grouped-query models and 40 per layer plus 30 for the 1.7B model, independent of batch size below the knee. Host dispatch time per kernel is $11.7\text{--}12.0\ \mu\text{s}$ in every cell below the knee (excluding the starred cells’ first passes, 17.4 and $14.6\ \mu\text{s}$), and host time outside synchronization waits (under the profiler) varies by less than 4% across batch sizes for each model with the dynamic cache (cell

means; second pass only for 135M at batch 1, whose first pass gave 45.3 versus 27.2 ms); extra sequences are therefore nearly free until GPU busy time catches up. For 135M, interpolating busy time between batch 32 and 128 puts that crossover near batch 70 at the profiled KV length of 280, a rough estimate from three points; the sweep’s latency already rises by batch 64. With the dynamic cache, Transformers’ attention-mask construction forces one GPU-to-CPU synchronization per step, so dispatch time tracks wall time by construction and the evidence for host-bound steps is the idle fraction and the batch-independent host time. The static cache has no such synchronization, yet at batch 1 its dispatch time (13.4 ms) equals its unsynchronized wall time.

Grouped-query attention falls back to the math backend on this build. The 15 extra kernels per layer in the two smaller models are all inside the SDPA operator (`aten::scaled_dot_product_attention`), which runs 16 kernels per call there versus one fused memory-efficient kernel for the 1.7B model. Transformers passes `enable_gqa=True` when no attention mask is needed; the Windows PyTorch wheel we used is not compiled with FlashAttention, and the memory-efficient kernel requires equal query and key/value head counts, so dispatch selects the math backend, which keeps intermediates in FP32 for BF16 inputs [20]. In isolation (KV length 300, batch 1) such a call takes 156–188 μ s of host time, versus 32 μ s and 3 kernels when K/V are repeated explicitly before a multi-head SDPA call. Taking that route in the model (`scripts/probe_sdpa_gqa.py`) cuts the batch-1 step from 20.0 to 15.9 ms for 135M (1,290 kernels) and from 21.5 to 17.1 ms for 360M (1,374 kernels), about 20%, with similar savings at batch 32 and 16. PyTorch documents `enable_gqa` as supported by its FlashAttention and math backends on CUDA [20], so this is a property of this build rather than of grouped-query attention.

Why the 1.7B model is faster, and where weight bytes matter. Without the fallback, step time per layer is 0.53, 0.53, and 0.50 ms for the three models, so the 1.7B model’s batch-1 advantage over 135M (11.9 vs. the probe’s 20.0 ms) splits roughly equally between the fallback (4.1 ms) and six fewer layers (4.0 ms); against 360M the split is 4.4 and 5.2 ms. Weight streaming dominates the 1.7B model’s GPU time: matrix–vector kernels take 81% of its kernel time at batch 1. Its step is nevertheless host-bound, because issuing 990 kernels takes the host about 11.8 ms against 7.2 ms of GPU work.

Above the knee. With idle time at 17–19% (23% for the static cache), these steps are GPU-bound. For the grouped-query models that time does not go to weights or the KV cache (1.1% and 3.2% of rated bandwidth for weights, 4.5% and 6.5% including the KV cache): about two-thirds of kernel time is element-wise copies and FP32 element-wise and matrix–vector kernels that occur only inside the math-backend attention call, while the linear layers’ BF16 matrix multiplications take under 8%. Without hardware counters we cannot say whether those kernels are bandwidth- or latency-limited. For the 1.7B model at batch 32, 29% of dynamic-cache kernel time is the `torch.cat` copies that grow the cache, which the static cache avoids, consistent with its 15% lower batch-32 latency in Table 3; at batch 1 it is slower mainly because it issues 154 more kernels per step, while attention over the full preallocated length adds only about 0.2 ms of GPU time.

Session offset. These runs came from a later session in which `generate()` took 11.6–13.2% longer than in Table 3 in every host-bound cell but only 0.7–1.2% longer in the five GPU-bound cells. The difference is thus host-side, and model ratios are unchanged (1.07 and 0.60 relative to 135M), so Table 4 should be read within itself. An earlier run of the same script, superseded because a background RGB-lighting application was using CPU, had identical kernel counts and GPU busy times within 1.3%, but its host-bound cells took 17–19% longer than in Table 3, versus 11.6–13.2% here; its data are in `results/profile_decode_run1/`. Without a Linux comparison we cannot separate framework overhead from Windows driver-model (WDDM) effects on per-kernel host cost.

4.6 Generation length

Figure 5 plots latency against generated token count for BF16 eager batch 1. Increasing generation length from 32 to 128 raised end-to-end latency by roughly $4\times$, while tokens-per-second stayed between 52 and 54 tok/s. Under this harness, wall-clock time tracks decode length more strongly than the modest prompt-length change (64 vs. 256).

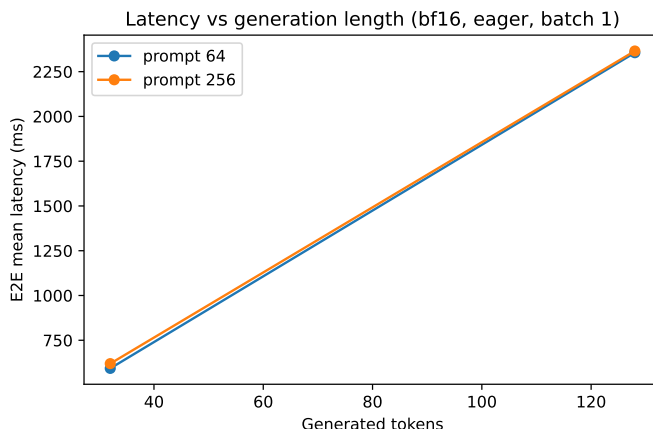


Figure 5: Latency versus generation length (BF16, eager, batch 1).

4.7 Compilation

Figure 6 reports eager/compile latency ratios. The median ratio was 1.015 (mean 1.027), ranging from 0.921 to 1.218. Thus `torch.compile` was approximately neutral on average for this model and measurement protocol, with occasional gains or regressions depending on the cell.

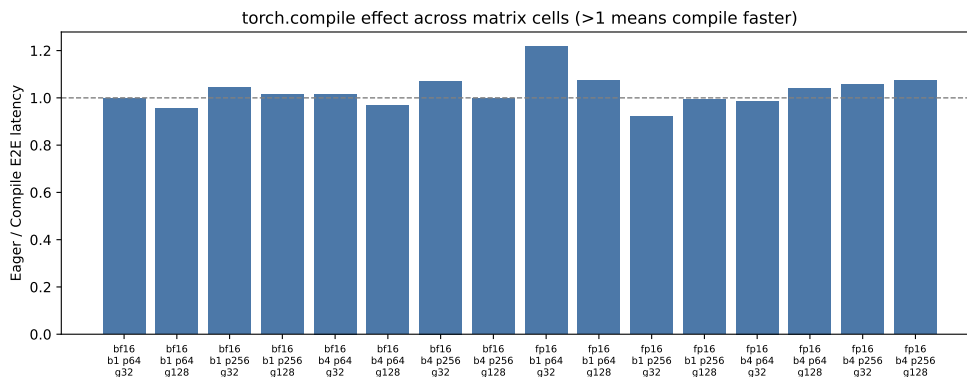


Figure 6: Eager/compile end-to-end latency ratios (> 1 means compile is faster).

5 Discussion and limitations

These measurements characterize one GPU, three models from a single family (with the factorial matrix on the smallest only), and an end-to-end `generate()` timer. They do not isolate time-to-first-token or

inter-token latency, do not evaluate weight quantization or continuous batching under load, and do not claim optimality of any backend. The batch-size sweep uses a single configuration and reports its first, excluded pass openly; its two retained passes agreed closely, but results on a shared desktop workstation remain sensitive to host state. The decode-step profile has two passes and three batch sizes per model, comes from a session that was slower on the host side, and lacks hardware counters, a Linux comparison, and a FlashAttention build (Section 4.5). We did not test the driver’s *Prefer No System Fallback* policy, so we do not report how the spilled 1.7B cell behaves when the fallback is disabled. The model-size comparison confounds parameter count with architecture (depth, width, and attention type), which is itself one of its findings, and a second GPU would be needed to separate properties of the models from properties of the RTX 5070. Within that scope, the following conclusions are stable in the data: BF16 reduced latency vs. FP16; for the 135M model, batching up to 32 raised throughput with no latency cost, and beyond batch 32 latency grew roughly linearly while throughput saturated; the batch size at which batching stops being free falls as the model grows; small-batch step time is set by host-side kernel issue rather than by weight bytes, with a grouped-query attention fallback costing the smaller models about a fifth of their step on this PyTorch build; a growing KV cache can inflate reserved memory several-fold and, under the Windows driver’s default system-memory fallback, spill into host memory without an error, which a static cache avoids; generation length dominated wall time; and `torch.compile` did not provide a consistent speedup.

6 Conclusion

We measured a full 32-cell inference matrix and a batch-size sweep on an RTX 5070 using a reproducible harness. The sweep shows a clear latency–throughput knee near batch 32 for the 135M model; the knee moves to batch 16 at 360M parameters and disappears at 1.7B, and for the largest model the choice of KV-cache implementation decides whether batch 64 fits on the device at all. The results emphasize workload- and precision-dependent behavior and caution against assuming that compilation alone improves short, small-model inference runs. Profiling shows that small-batch decoding here is host-bound, at about $12\ \mu\text{s}$ of host time per kernel; natural next steps are CUDA Graphs or other ways of reducing per-step kernel issue, a PyTorch build with FlashAttention, and a Linux comparison on the same GPU.

Code and data availability

The harness, configurations, analysis scripts, and aggregated results are available at <https://github.com/benkarlsberg/GPU-Optimized-Transformer-Inference>.

Acknowledgments

The author conceived and designed this study. AI assistants were used to help implement the benchmarking code, run the experiments, and draft the manuscript under the author’s direction. The author reviewed all code, results, and references and takes full responsibility for the content.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

- [2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- [3] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4895–4901, 2023.
- [5] L. Ben Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf. SmolLM: Blazingly fast and remarkably powerful. Hugging Face blog, <https://huggingface.co/blog/smollm>, 2024.
- [6] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.
- [7] J. Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [8] K. T. Chitty-Venkata, S. Raskar, B. Kale, F. Ferdaus, A. Tanikanti, K. Raffanetti, V. Taylor, M. Emani, and V. Vishwanath. LLM-Inference-Bench: Inference benchmarking of large language models on AI accelerators. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1362–1379, 2024.
- [9] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [11] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [12] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [13] D. Kalamkar et al. A study of BFLOAT16 for deep learning training. arXiv:1905.12322, 2019.
- [14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [15] P. J. Maliakel, S. Ilager, and I. Brandic. Characterizing LLM inference energy-performance tradeoffs across workloads and GPU scaling. In *2026 IEEE 26th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 33–43, 2026.

- [16] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.
- [17] NVIDIA Corporation. Compare current and previous GeForce series of graphics cards. Specifications table, GeForce RTX 5070 column. <https://www.nvidia.com/en-us/geforce/graphics-cards/compare/>, accessed 2026-09-25.
- [18] NVIDIA Corporation. System memory fallback for Stable Diffusion. NVIDIA Support answer 5490, https://nvidia.custhelp.com/app/answers/detail/a_id/5490, accessed 2026-09-25.
- [19] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- [20] PyTorch Contributors. `torch.nn.functional.scaled_dot_product_attention`. PyTorch 2.11 documentation, https://docs.pytorch.org/docs/2.11/generated/torch.nn.functional.scaled_dot_product_attention.html, accessed 2026-09-25.
- [21] P. G. Recasens, Y. Zhu, C. Wang, E. K. Lee, O. Tardieu, A. Youssef, J. Torres, and J. Ll. Berral. Towards Pareto optimal throughput in small language model serving. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys)*, pages 144–152, 2024.
- [22] P. G. Recasens, F. Agullo, Y. Zhu, C. Wang, E. K. Lee, O. Tardieu, J. Torres, and J. Ll. Berral. Mind the memory gap: Unveiling GPU bottlenecks in large-batch LLM inference. In *2025 IEEE 18th International Conference on Cloud Computing (CLOUD)*, pages 277–287, 2025.
- [23] V. J. Reddi et al. MLPerf inference benchmark. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [24] N. Shazeer. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150, 2019.
- [25] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [26] H. Wang, X. Xiao, M. Yan, Z. Zhu, D. Han, D. Wang, W. Li, X. Ye, C. Hu, H. Chen, and G. Sun. A systematic characterization of LLM inference on GPUs. arXiv:2512.01644, 2025.
- [27] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [28] T. Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.
- [29] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.